

Small-Collection Vector Search on CPU and Portable GPU: Completed-Call Performance, Resource Cost, and Mutation Behavior

Akshat Singh Kushwaha
akshatsingh14372@outlook.com
<https://github.com/a3ro-dev/qenlo>

September 2026

Abstract

Many embedded and desktop applications search thousands rather than millions of vectors. At that scale, fixed costs, memory duplication, filter preparation, and the first query after a mutation can matter more than asymptotic index complexity. We study Qenlo, a Rust vector store with one canonical record model and optional exact CPU, portable WGPU, and PyTorch execution. The target regime is 10^3 to 10^5 vectors, dimensions 128–768, query batches 1–64, and filtered as well as unfiltered search. All comparisons time a completed host call through ordered identifiers and distances; correctness is checked independently against an FP64 oracle.

On an RTX 4090, exact WGPU search over 100k 768-dimensional vectors reaches 0.897 ms P95 for batch one and $k = 1$, versus 9.409 ms for the native CPU and 14.800 ms for FAISS Flat. PyTorch CUDA is faster at 0.671 ms. With batch eight, $k = 64$, and 10% eligibility, WGPU reaches 0.896 ms, FAISS Flat 3.055 ms, and PyTorch CUDA 0.384 ms. Every corrected row has oracle recall 1.0. The lane-minimum selector optimization is not a universal win: across 12 qualified before/after WGPU pairs, it wins five and loses seven. In-place GPU append and live-mask updates avoid a rebuild in the measured mutation cells, while reopen still rebuilds resident state. The campaign retains unavailable hardware, three Chroma correctness disqualifications, two invalid-harness rows, and four pre-fix admission failures. Final known cloud spend was \$0.9401. These results support a small-collection portable-GPU path and explicit resource admission. They do not establish mobile performance or a universally fastest database.

1 Introduction

The usual vector-search story starts with scale: as N grows, scan cost grows, so an approximate index becomes attractive. That is true, but it skips the regime many applications actually ship. A phone, desktop tool, local retrieval feature, or per-user collection often holds between 1k and 100k vectors. Here a search implementation pays for predicate evaluation, dispatch, synchronization, top- k selection, and result conversion on every call. A fast kernel can still lose once those costs are included.

This paper asks a narrower question: how should one durable vector-store core execute exact filtered search over small collections when CPU, portable GPU, and an application’s existing tensor runtime are available? Resource use is part of the answer. We report latency with process RSS, Qenlo-owned accelerator allocations, scratch, transfers, build time, and post-mutation preparation where the harness can attribute them. We do not collapse these quantities into one efficiency score.

The study makes four contributions. First, it defines one correctness and persistence boundary across embedded and accelerated builds. Second, it evaluates a portable WGSL top- k selector in which each lane retains its minimum and only the winning lane rescans its stripe. Third, it measures in-place mutation updates and a generation-bound optional tensor index. Fourth, it publishes a failure-preserving matrix over synthetic and retained real embeddings, with Chroma, FAISS Flat, NumPy, USearch, and PyTorch comparisons.

The main finding is mixed. Portable GPU execution is useful at 100k vectors, including batch-one calls, but the selector change regresses more paired cells than it improves. PyTorch CUDA wins the two corrected 768-dimensional cells, while it carries a larger runtime and is not the durable source of truth. That result argues for measured deployment profiles, not one backend everywhere.

2 System and contracts

2.1 One canonical core, two profiles

Qenlo stores normalized FP32 vectors, unsigned public identifiers, user and timestamp metadata, tombstones, and a monotonically increasing generation in one canonical Rust store. Distance is cosine distance. Results use one total order: ascending distance, then ascending identifier. Atomic batches publish only after validation and durable staging; stale derived plans fail closed.

The embedded profile keeps CPU exact search, bounded scratch, portable snapshots, and the language bridge needed by the host application. GPU support is an optional build feature. The accelerated profile adds WGPU, richer diagnostics, bulk ingestion, and optional PyTorch integration. These profiles select features around the same records and persistence semantics; they are not separate databases.

Execution policy is explicit. CPU mode never initializes a GPU. Required-GPU mode fails on initialization, capacity, or device loss instead of falling back. Automatic mode reports the requested and actual route plus any fallback reason. A load budget covers the canonical rows, an aligned CPU scan view, metadata-index allowance, and durable-write admission. GPU admission covers resident vectors, identifiers, masks or eligible-row lists, selector scratch, query data, and readback staging.

2.2 Portable exact GPU execution

The WGPU backend uses one WGSL implementation across Vulkan, Metal, and DX12. Vectors stay resident when the generation is prepared. A query scores every eligible row, produces bounded candidates, and returns host-visible identifiers and distances. The evaluated selector candidate retained each lane’s current minimum and rescanned only the lane that supplied the previous winner. Empty eligibility, partial workgroups, maximum k , and distance/identifier ties used the same ordering as the CPU path. Because the campaign found seven regressions in 12 qualified pairs, the final source keeps the simpler full-rescan selector.

This optimization reduces redundant stripe scans, but it adds lane state and changes synchronization behavior. We therefore keep it only as a measured implementation change. Table 2 shows that it is not uniformly faster.

2.3 Mutation and tensor integration

Adds append resident rows when capacity permits. Deletes update a live mask. Each derived update is bound to the canonical generation, so a search cannot combine vectors from one generation with liveness from another. A full rebuild remains the bounded fallback when capacity, layout, or device state prevents an incremental update.

The Python `TorchIndex` is an optional immutable tensor view. It accepts contiguous arrays or tensors, supports CPU, CUDA, and MPS device selection, executes query batches as matrix operations, and returns tensors without a JSON round trip. A canonical collection snapshot carries its generation and filter identity. A mutation makes the snapshot stale; callers must reject or rebuild it. PyTorch remains a desktop extra and is imported lazily. It is not required by mobile artifacts and is not a second durable store.

3 Experimental method

3.1 Workloads and engines

The common sweep covers 1k, 10k, and 100k rows; dimensions 128 and 384; batches one and 16; and unfiltered or 1%-eligible predicates. A reference RTX 4090 adds 5k, 25k, and 50k cells, batch 64, $k \in \{1, 10, 64\}$, compound filters, lifecycle measurements, and a retained 100k-by-384 AG News embedding fixture. A corrected supplement covers 100k-by-768 at batch one, $k = 1$, unfiltered and batch eight, $k = 64$, 10% eligible.

The engines are the frozen pre-change Qenlo CPU/WGPU revision, the revised native CPU/WGPU implementation, NumPy, FAISS Flat, PyTorch CPU/CUDA, Chroma HNSW, and USearch where the harness supports the workload. We do not equate approximate and exhaustive rows. Chroma and USearch qualify only when independent FP64-oracle recall meets the predeclared target.

3.2 Timing and qualification

Completed-call time starts before the host search call and ends after ordered identifiers and distances are visible to the host. Device-resident PyTorch timing is reported separately and is never substituted for completed-call latency.

Table 1: Completed-call latency on one RTX 4090/Vulkan host. Times are milliseconds. Recall is against an independent FP64 oracle; samples are completed calls for native engines and complete repetitions for external replays.

Workload	Engine	P50	P95	Recall	Samples
100k, B=1, k=1, 100%	Qenlo CPU	9.010	9.409	1.0	192
100k, B=1, k=1, 100%	Qenlo WGPU	0.870	0.897	1.0	192
100k, B=1, k=1, 100%	FAISS Flat	14.483	14.800	1.0	3
100k, B=1, k=1, 100%	NumPy	14.507	15.343	1.0	3
100k, B=1, k=1, 100%	Torch CPU	18.324	18.695	1.0	3
100k, B=1, k=1, 100%	Torch CUDA	0.654	0.671	1.0	3
100k, B=8, k=64, 10%	Qenlo CPU	27.402	28.599	1.0	24
100k, B=8, k=64, 10%	Qenlo WGPU	0.842	0.896	1.0	24
100k, B=8, k=64, 10%	FAISS Flat	2.716	3.055	1.0	3
100k, B=8, k=64, 10%	NumPy	5.972	6.202	1.0	3
100k, B=8, k=64, 10%	Torch CPU	2.847	2.936	1.0	3
100k, B=8, k=64, 10%	Torch CUDA	0.370	0.384	1.0	3

Corpus load, index construction, and warmup are outside the query interval. Engine order rotates, raw samples are retained, and reported P50/P95/P99 values are lower medians of per-run nearest-rank percentiles.

Native rows contain 24–192 completed calls depending on batch size. External replay rows use three complete repetitions. That is enough to expose large effects and failures, but too little for narrow confidence intervals. We report exact sample counts and avoid significance claims.

The corrected 768-dimensional supplement uses source archive SHA-256 `bb195fed...eb19`. Earlier current/native rows use `730c0500...e3`; the frozen baseline archive is `f37e744c...a45d`. The working tree was dirty, so the archive hashes, not the Git commit alone, identify executable source.

3.3 Campaign controls

The Runpod controller checked spend since midnight in Asia/Calcutta, reserved \$1, calculated worst-case exposure before each creation, recorded pod identifiers immediately, and started an independent deletion watchdog. Ten primary configurations were attempted across RTX 2000 Ada, L4, A40, RTX 3090/4090, and A6000 families. Unavailable offers remain in the matrix instead of being replaced. Deeper work ran only on a reference RTX 4090. At completion, the API returned zero pods and final known daily spend was \$0.9400778694. Billing lag remains possible; no campaign volume or disk was retained.

4 Results

4.1 The corrected 100k-by-768 cells

Table 1 is the clearest result in the new campaign. All 12 rows complete and have oracle recall 1.0. In the unfiltered batch-one cell, native WGPU is $10.49\times$ faster than native CPU and $16.50\times$ faster than FAISS Flat at P95. PyTorch CUDA is another $1.34\times$ faster than WGPU. In the 10%-eligible batch-eight cell, WGPU is $31.91\times$ faster than CPU and $3.41\times$ faster than FAISS Flat, while PyTorch CUDA is $2.33\times$ faster than WGPU.

The resource result is less flattering. In the unfiltered cell, WGPU reports 311.2 MB of Qenlo-owned accelerator allocation and a 1.204 GB process RSS high-water mark, versus 665.8 MB RSS for CPU. The call uploads 403,136 bytes and reads back 48 bytes. With batch eight and $k = 64$, owned accelerator allocation is 314.9 MB, upload is 517,120 bytes, and readback is 65,792 bytes. These are different scopes: process RSS is a process-wide high-water mark, while accelerator allocation is backend-attributable. We do not add or divide them into one score.

Phase timing explains the second cell. Median device scoring is 0.072 ms, selection is 0.613 ms, backend execution is 0.826 ms, and completed-call P95 is 0.896 ms. Selection, synchronization, and readback dominate arithmetic. This is the case for a hierarchical selector only if its extra scratch and dispatch can beat that 0.613 ms in a matched run.

4.2 Before/after selector behavior

Table 2 reports every qualified pair between the frozen WGPU revision and the lane-minimum revision. A ratio above one favors the revision. Five of 12 cells improve; seven regress. Gains reach $1.295\times$ on an A40, while the worst

observed ratio is $0.617\times$ on the RTX 4090 50k compound-filter cell. The same code can move in opposite directions across A40 hosts and workloads.

Table 2: Frozen-baseline P95 divided by revised-WGPU P95. Values above one are improvements.

Configuration	Workload	Ratio
RTX 2000 Ada, EU-RO	1k, 128D, B=1, full	0.777
A40, CA-MTL	1k, 128D, B=1, full	0.881
A40, CA-MTL	10k, 384D, B=1, 1%	0.855
A40, CA-MTL	100k, 128D, B=1, 1%	1.043
A40, EU-SE	1k, 128D, B=1, full	1.295
A40, EU-SE	10k, 384D, B=1, 1%	1.096
A40, EU-SE	100k, 128D, B=1, 1%	1.205
RTX 4090, EU-RO	1k, 128D, B=1, full	0.840
RTX 4090, EU-RO	10k, 384D, B=1, 1%	0.853
RTX 4090, EU-RO	100k, 128D, B=1, 1%	1.073
RTX 4090, deep	5k, 384D, B=8, 10%	0.889
RTX 4090, deep	50k, 384D, B=1, compound	0.617

The acceptance rule was to retain complexity only when measured benefit justifies it. These data do not justify making the lane-minimum selector an unconditional replacement. The implementation therefore reverts the revised selector and retains its measurements as a negative result. A future bounded hierarchical merge or dispatch rule needs matched evidence on the losing cells before it enters the durable path.

4.3 Mutation and reopen cost

On the RTX 4090 reference workload, add-one then search reaches 0.598 ms P95; delete-one then search reaches 0.631 ms. Small add and delete batches reach 0.740 and 0.819 ms. Rebuild fraction is zero in all four cells, and deletion checks pass. Mutation publication itself costs about 22 ms P95 in these durable runs. Reopen is different: first search takes 101.296 ms and rebuild fraction is one. Incremental resident updates solve the warm mutation case; they do not eliminate restart preparation.

4.4 Chroma target and qualification

In the seven qualified paired cells, revised WGPU is faster than Chroma HNSW at the completed-call boundary; ratios range from $3.700\times$ to $3233.136\times$. This is a scoped result, not a product-wide claim. Chroma includes its own API and filtering path, uses approximate HNSW, and three attempted Chroma configurations fail the predeclared recall gate before held-out timing: two at `ef_search=128` and one at 512. The disqualified rows remain visible. The result supports “Qenlo beat Chroma in these qualified cells,” not “Qenlo is the fastest database.”

5 Failures and negative results

The campaign matrix has 182 rows: 131 completed, 42 unavailable primary-configuration rows, seven engine failures, and two invalid-harness rows. Four of the seven engine failures are 100k-by-768 admission rejections from the pre-fix source. The benchmark accepted a 2 GiB vector budget but constructed the in-memory collection with the 512 MiB default storage admission. The final supplement adds an explicit in-memory constructor option and forwards the benchmark budget. A regression test proves that the supplied limit governs atomic batches. We retain both the failures and the corrected-source hash.

The first 768-dimensional supplement also passed internal cell labels to the benchmark as positional arguments. No benchmark executed. Those two rows are marked `invalid_harness`; they are not counted as performance failures. This distinction matters because a green controller exit only proves that the script reached its end. Artifact status, hashes, summaries, and oracle qualification determine whether a measurement is usable.

6 Deployment implications

For embedded builds, CPU exact search remains the default because it has no accelerator residency, driver initialization, or desktop tensor dependency. A mobile application may enable WGPU only after measuring completed-call latency and peak memory on the physical device. This campaign has no Android or iOS measurements, so it cannot select a mobile default.

For accelerated desktop builds, exact WGPU is a reasonable measured option near 100k rows. PyTorch CUDA is faster in the two 768-dimensional cells, but bundling PyTorch only for search adds installation and runtime cost. The tensor path makes sense when the application already owns a compatible embedding runtime or needs tensor-native batches. FAISS Flat remains a useful independent exhaustive baseline. Chroma remains an approximate database comparison with different persistence and filtering boundaries.

The simplest current policy is explicit backend selection plus transparent automatic fallback. The matrix is too small for a learned router. Fixed choices should use collection size, dimension, batch, eligible fraction, and device class; any selector-specific rule must be validated on the actual adapter.

7 Related work

FAISS provides optimized exhaustive and approximate CPU/GPU search [3, 6]. HNSW trades recall for graph-traversal work [7]. DiskANN and SPANN address much larger storage-aware regimes [2, 10]. Filtered-DiskANN and ACORN modify graph access under predicates [4, 8]. Recent filtered-ANN studies show that selectivity, correlation, and implementation change the best method [1, 5, 9].

Our contribution is not a new asymptotic nearest-neighbor algorithm. It is a measured implementation boundary for exact search over small collections: one canonical durable store, bounded derived state, portable GPU selection, completed-call timing, mutation behavior, and explicit failure accounting.

8 Limitations

The current cloud evidence covers NVIDIA Vulkan hosts, not Metal, DX12, Android, iOS, or phone Linux. Ten primary configurations were attempted, but only the available RTX 2000 Ada, A40, and RTX 4090 hosts produced measurements. External engines use three repetitions in the corrected supplement, which limits uncertainty estimates. Synthetic vectors dominate the matrix; the retained AG News fixture covers one 384-dimensional workload. Filters are controlled but do not span realistic vector/metadata correlation. There is no power measurement.

Process RSS is a high-water mark and cannot be assigned cleanly to one library. GPU allocation is Qenlo-owned rather than physical VRAM telemetry. The WGPU and PyTorch paths use different runtime stacks. Chroma’s database/API boundary is broader than a raw matrix operation. The paper reports those boundaries instead of treating every row as interchangeable.

The release is also incomplete as a platform claim until Apple device/simulator artifacts and Android packages are built in CI and exercised on physical devices. Cloud NVIDIA results cannot fill that gap.

9 Conclusion

Small collections are large enough for accelerator execution to matter and small enough for fixed costs to decide the result. On one RTX 4090, Qenlo WGPU completes exact 100k-by-768 search in about 0.9 ms P95 and beats the tested CPU, NumPy, and FAISS Flat paths. PyTorch CUDA is faster in both corrected 768-dimensional cells. The lane-minimum selector itself wins five and loses seven qualified baseline pairs, so it is not ready as an unconditional optimization.

The practical result is a narrower product direction. Keep one canonical Rust store. Make CPU, required GPU, automatic fallback, and tensor execution explicit. Admit memory before the first query. Update resident state by generation after mutations. Publish every failure beside the wins. The remaining claim is deliberately plain: Qenlo now has credible small-collection acceleration evidence on the tested desktop GPUs. Mobile performance and a universal fastest-database claim remain unproven.

Reproducibility statement

The repository retains controller ledgers, source archive hashes, exact workload scripts, environment captures, raw samples, engine summaries, independent oracle checks, invalid and unavailable rows, the 182-row CSV/JSON matrix, and the table generator used by this paper. Appendix A maps headline claims to artifacts and gives reproduction commands.

References

- [1] Abylay Amanbayev, Brian Tsan, Tri Dang, and Florin Rusu. Filtered approximate nearest neighbor search in vector databases: System design and performance analysis. *arXiv preprint arXiv:2602.11443*, 2026. URL <https://arxiv.org/abs/2602.11443>.
- [2] Qi Chen, Bing Zhao, Haidong Wang, Mingqin Li, Chuanjie Liu, Zengzhong Li, Mao Yang, and Jingdong Wang. SPANN: Highly-efficient billion-scale approximate nearest neighborhood search. In *Advances in Neural Information Processing Systems*, volume 34, pages 5199–5212, 2021. URL https://proceedings.neurips.cc/paper_files/paper/2021/hash/299dc35e747eb77177d9cea10a802da2-Abstract.html.
- [3] Matthijs Douze, Alexandr Guzhva, Chengqi Deng, Jeff Johnson, Gergely Szilvasy, Pierre-Emmanuel Mazaré, Maria Lomeli, Lucas Hosseini, and Hervé Jégou. The Faiss library. *arXiv preprint arXiv:2401.08281*, 2024. URL <https://arxiv.org/abs/2401.08281>.
- [4] Siddharth Gollapudi, Neel Karia, Varun Sivashankar, Ravishankar Krishnaswamy, Nikit Begwani, Swapnil Raz, Yiyong Lin, and Yin Zhang. Filtered-DiskANN: Graph algorithms for approximate nearest neighbor search with filters. In *Proceedings of the ACM Web Conference 2023*, pages 3406–3416, 2023. doi: 10.1145/3543507.3583552.
- [5] Patrick Iff, Paul Brügger, Marcin Chrapek, David Kochergin, Maciej Besta, and Torsten Hoeffler. Benchmarking filtered approximate nearest neighbor search algorithms on transformer-based embedding vectors. In *Proceedings of the 49th International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 610–622, 2026. doi: 10.1145/3805712.3809731. URL <https://doi.org/10.1145/3805712.3809731>.
- [6] Jeff Johnson, Matthijs Douze, and Hervé Jégou. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 7(3):535–547, 2021. doi: 10.1109/TBDDATA.2019.2921572.
- [7] Yu. A. Malkov and D. A. Yashunin. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 42(4):824–836, 2020. doi: 10.1109/TPAMI.2018.2889473.
- [8] Liana Patel, Peter Kraft, Carlos Guestrin, and Matei Zaharia. ACORN: Performant and predicate-agnostic search over vector embeddings and structured data. *Proceedings of the ACM on Management of Data*, 2(3), 2024. doi: 10.1145/3654923.
- [9] Jiayang Shi, Yuzheng Cai, and Weiguo Zheng. Filtered approximate nearest neighbor search: A unified benchmark and systematic experimental study. *arXiv preprint arXiv:2509.07789*, 2025. URL <https://arxiv.org/abs/2509.07789>.
- [10] Suhas Jayaram Subramanya, Fnu Devvrit, Harsha Vardhan Simhadri, Ravishankar Krishnaswamy, and Rohan Kadekodi. DiskANN: Fast accurate billion-point nearest neighbor search on a single node. In *Advances in Neural Information Processing Systems*, volume 32, 2019. URL <https://proceedings.neurips.cc/paper/2019/hash/09853c7fb1d3f8ee67a61b6bf4a7f8e6-Abstract.html>.

A Artifact and protocol details

A.1 Claim-to-artifact ledger

The table below lists the evidence required for each headline claim. The machine-readable version is `paper/tables/claim-to-artifact.csv`.

Table 3: Headline claims and authoritative local evidence.

ID	Claim	Evidence
C1	All corrected 100k-by-768 rows completed at oracle recall 1.0.	<code>performance-matrix.csv</code> ; corrected supplement archive and 74-entry checksum manifest.
C2	WGPU P95 is 0.897 ms for 100k-by-768, batch one, $k = 1$.	Corrected configuration, workload <code>d4</code> , engine <code>current-gpu</code> .
C3	PyTorch CUDA is faster than WGPU in both corrected 768-dimensional cells.	Corrected <code>d4/d5</code> WGPU and <code>torch-cuda</code> rows.
C4	The revised selector wins five and loses seven of 12 baseline pairs.	Qualified paired-comparison section of <code>performance-report.md</code> .
C5	Current mobile performance is unmeasured.	Campaign scope and absence of Android/iOS host records in the matrix.
C6	Final known spend is \$0.9400778694 and no pods remain.	Final billing capture, deletion records, and terminal pod-list check.

A.2 Artifact locations and hashes

The campaign root is `research/artifacts/runpod-small-2026-09-05`. Its reduced outputs are:

- `report/performance-matrix.csv`, one flat row per engine/workload/status;
- `report/performance-matrix.json`, the machine-readable equivalent;
- `report/performance-report.md`, qualified pair summaries;
- `ledger.jsonl`, creation, polling, billing, deletion, and cleanup events.

The original current source archive is identified by SHA-256 prefix `730c0500`; it is preserved under a hash-qualified filename after the admission defect was found. The corrected source archive uses prefix `bb195fed`, and the frozen pre-change archive uses `f37e744c`. The corrected supplement archive uses `9e322d61`; all 74 manifest entries were verified locally. Full hashes are recorded in `paper/REPRODUCE.md` and the campaign manifests.

A.3 Correctness rules

The oracle normalizes finite inputs and computes cosine distance with FP64 accumulation. Each engine receives the same corpus split, queries, filter, and k . Results are compared under ascending distance then ascending unsigned identifier. Tied cutoffs accept only the deterministic members selected by that order. Rows with non-finite outputs, missing identifiers, stale generation, incorrect deletion visibility, or recall below the predeclared target do not qualify.

Native CPU and WGPU are exhaustive FP32-storage engines; exhaustive does not mean bit-identical to the FP64 oracle. Chroma and USearch are approximate and qualify by measured recall. Device-resident tensor timing is separate from the host completed-call field.

A.4 Lifecycle semantics

An add or delete first changes canonical state and generation. A resident WGPU update is published only for that generation. Searches holding an older plan fail or rebuild; they never mix vector and liveness generations. Deletion cells explicitly test that the removed identifier is absent. Reopen reconstructs resident derived state from the durable canonical snapshot and WAL.

A.5 Failure taxonomy

Unavailable. The requested GPU/data-center/cloud configuration could not be provisioned at the live quote. No substitute hardware is assigned to that row.

Failed. The engine or correctness gate ran and rejected the cell. This includes Chroma recall-gate failures and pre-fix storage admission.

Invalid harness. The orchestration command was malformed, so no measurement is eligible. Cost and reason remain in the ledger.

Completed, unqualified. The engine produced output but failed a declared semantic criterion. Such a row remains visible and is excluded from paired claims.

A.6 Resource scopes

`peak_process_rss_bytes` is a process high-water mark. `qenlo_allocation_bytes` is backend-attributable owned memory where available. Upload and readback fields count per completed call. Build time records binary or index preparation outside query latency. Missing measurements are empty, not zero. A zero is emitted only when the measured scope performed no work.

A.7 Reproduction commands

Local correctness work should use one build job and at most two numerical threads:

```
set CARGO_BUILD_JOBS=1
set RAYON_NUM_THREADS=2
cargo test -p qenlo --lib -- --test-threads=1
cargo test -p qenlo-bench --no-default-features -- --test-threads=1
py -3 scripts/analyze_small_campaign.py \
    --campaign research/artifacts/runpod-small-2026-09-05 \
    --output research/artifacts/runpod-small-2026-09-05/report
py -3 scripts/generate_small_paper_tables.py
```

The Runpod controller is `scripts/runpod_small_campaign.ps1`. It refuses a campaign when the daily operational allowance or cleanup checks fail. Re-running paid work should use new result directories and current live prices; retained artifacts must not be overwritten.

B Additional result notes

The real-embedding cell uses a retained 100k-by-384 AG News fixture with provenance and checksum, not a remote dataset downloaded at run time. This avoids silently changing preprocessing, query partition, identifiers, or normalization. Future campaigns should publish that exact immutable fixture to content-addressed object storage and download it on the pod; the present run transferred the checksummed prepared artifact because no immutable public URL existed.

The matrix does not include macOS MPS measurements. The tensor API supports explicit MPS selection in code, but support and performance are separate claims. Likewise, a Kotlin/JVM bridge is not evidence of an Android package, and an Apple simulator artifact is not an iOS-device result. Those platform rows stay open until their native artifacts execute on the named targets.